

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	G. Naveen Kumar Reddy	Reg .No.:	192525288
Department:	AIML	Date:	

Code of Conduct:

I G. Naveen Kumar Reddy (192525288) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Naveen Kumar Reddy (192525288)

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Register Transfer, ALU Operations and Data Flow

Aim

To design and simulate a register-transfer level (RTL) system that performs ADD, SUB, AND and OR operations using multiple registers and a common ALU, and to analyze the sequence of micro-operations and the role of the ALU.

Tools Used

- Logisim / Digital Works (for schematic-level simulation of registers, bus and ALU)
- Python 3 (for functional/behavioral simulation of the same RTL sequence)

System Design

The datapath consists of three general purpose registers (R1, R2, R3), a common internal bus, and an ALU that receives its two operands from bus-latched buffers (A and B) and writes its result back onto the bus. Each register has an input control line (Ri_in) and an output control line (Ri_out) that connect it to the bus.

Component	Function
R1, R2, R3	General purpose registers, each with in/out gating to the bus
Bus	Common path carrying data between registers, ALU and memory
A, B (ALU latches)	Hold the two ALU operands for one clock period
ALU	Performs ADD, SUB, AND, OR on the values latched in A and B
Z (output latch)	Temporarily holds the ALU result before it is gated onto the bus

Register Transfer Sequence (Micro-operations)

Example: $R3 \leftarrow R1 + R2$, executed in four control steps:

Step	Micro-operation	Control Signals Active
T0	$A \leftarrow R1$	R1_out, A_in
T1	$B \leftarrow R2$	R2_out, B_in
T2	$Z \leftarrow A + B$ (ALU op = ADD)	ALU_ADD, Z_in
T3	$R3 \leftarrow Z$	Z_out, R3_in

The same 4-step pattern is reused for SUB (ALU_SUB), AND (ALU_AND) and OR (ALU_OR) simply by changing the ALU opcode asserted at T2, which shows the ALU acting as the single shared processing element for every arithmetic/logic instruction.

Behavioral Simulation Program (Python)

```
class Register:
    def __init__(self, name, value=0):
        self.name, self.value = name, value
```

```

def ALU(op, a, b):
    ops = {
        'ADD': a + b, 'SUB': a - b,
        'AND': a & b, 'OR' : a | b
    }
    return ops[op]

R1, R2, R3 = Register('R1', 12), Register('R2', 5), Register('R3')

for op in ['ADD', 'SUB', 'AND', 'OR']:
    A, B = R1.value, R2.value          # T0, T1: A<-R1 , B<-R2
    Z = ALU(op, A, B)                  # T2  : Z <- A op B
    R3.value = Z                       # T3   : R3 <- Z
    print(f'R3 <- R1 {op} R2 => {R3.value}')

```

Observation (Output)

Operation	R1	R2	R3 = Result	Remarks
ADD	12	5	17	Arithmetic addition via ALU adder
SUB	12	5	7	2's complement subtraction (R1 + (-R2))
AND	1100	0101	0100	Bitwise AND - useful for masking
OR	1100	0101	1101	Bitwise OR - useful for setting bits

Analysis

Every operation follows the same four-step register-transfer pattern (source-out, latch, ALU-compute, destination-in); only the ALU opcode changes. This confirms that the ALU is the single shared functional unit through which all arithmetic and logic transfers must pass, and that a common bus economizes wiring at the cost of one register transfer per clock cycle (only one source and one destination can use the bus at a time).

Experiment 2: Single-Bus vs Multi-Bus Organization

Aim

To model a computer system using single-bus and multi-bus (three-bus) organization, perform register-memory-I/O transfers on each, and compare their timing and efficiency.

Tools Used

- Logisim (structural model of single-bus and 3-bus datapaths)
- Python 3 (cycle-count simulation for timing comparison)

Single-Bus Organization

All units (registers, ALU, Memory Address/Data Registers, I/O) share ONE common bus. Only one transfer can occur per clock cycle because only one source can drive the bus and only one destination can be enabled at a time.

Cycle	Transfer on the single bus
1	MAR <- R1 (address out)
2	MDR <- Memory[MAR] (read)
3	R2 <- MDR
4	A <- R2, B <- R3 cannot occur together - needs 2 cycles

Multi-Bus (3-Bus) Organization

Three buses (Bus-A, Bus-B feeding ALU inputs, and Bus-C carrying the ALU/memory result) allow two source registers to be read and one result to be written back in the SAME clock cycle.

Cycle	Transfer on Bus-A	Transfer on Bus-B	Transfer on Bus-C
1	R1 -> ALU (A)	R2 -> ALU (B)	ALU result -> R3
2	R4 -> MAR	-	Memory[MAR] -> MDR

Timing Comparison Simulation (Python)

```
# Sequence: R3 <- R1 + R2 ; R5 <- Memory[R4]
single_bus_cycles = 4 + 3      # 4 for ADD sequence, 3 for the load
multi_bus_cycles  = 1 + 2      # ADD in 1 cycle, load in 2 cycles

speed_gain = single_bus_cycles / multi_bus_cycles
print('Single-bus cycles :', single_bus_cycles)
print('Multi-bus cycles  :', multi_bus_cycles)
print(f'Efficiency gain   : {speed_gain:.2f}x')
```

Output: Single-bus cycles : 7 Multi-bus cycles : 3 Efficiency gain : 2.33x

Comparison Table

Parameter	Single-Bus	Multi-Bus (3-Bus)
Simultaneous transfers	1	Up to 3
Hardware cost	Low	Higher (more bus drivers/lines)
Control complexity	Simple	More complex sequencing/arbitration
Cycles for $R3 \leftarrow R1 + R2$	4	1
Typical use	Low-cost microcontrollers	High-performance CPUs

Analysis

The multi-bus datapath removes the bus-contention bottleneck of the single-bus design by giving the ALU dedicated input paths and a dedicated write-back path, so the same ADD sequence collapses from 4 cycles to 1. The gain comes at the cost of extra bus drivers, multiplexers and more complex control logic, which is why multi-bus organization is favoured in high-performance CPUs while single-bus remains attractive for low-cost, low-power designs.

Experiment 3: 5-Stage Instruction Pipeline - Speedup & Throughput

Aim

To simulate a 5-stage pipeline (Fetch - Decode - Execute - Memory - Write-back), execute a sequence of instructions, and measure speedup and throughput against a non-pipelined execution of the same sequence.

Tools Used

- Python 3 (pipeline cycle simulator)
- MARS / Logisim (optional structural verification of stage registers IF/ID, ID/EX, EX/MEM, MEM/WB)

Pipeline Execution Diagram (5 instructions, ideal / no hazards)

Instr \ Cycle	1	2	3	4	5	6	7	8	9
I1	F	D	E	M	W				
I2		F	D	E	M	W			
I3			F	D	E	M	W		
I4				F	D	E	M	W	
I5					F	D	E	M	W

With n instructions and $k = 5$ stages, the pipeline needs $(k + n - 1)$ cycles instead of $(n \times k)$ cycles for sequential (non-pipelined) execution.

Simulation Program (Python)

```
k = 5                # number of pipeline stages
n = 5                # number of instructions

non_pipelined_cycles = n * k
pipelined_cycles      = k + (n - 1)

speedup    = non_pipelined_cycles / pipelined_cycles
throughput = n / pipelined_cycles    # instructions per cycle

print('Non-pipelined cycles :', non_pipelined_cycles)
print('Pipelined cycles      :', pipelined_cycles)
print(f'Speedup              : {speedup:.2f}')
print(f'Throughput           : {throughput:.2f} instr/cycle')
```

Result Table

Metric	Non-Pipelined	Pipelined	Improvement
Total cycles (n = 5)	25	9	2.78x fewer cycles

Metric	Non-Pipelined	Pipelined	Improvement
Speedup	1x (baseline)	2.78x	-
Throughput (instr/cycle)	0.20	0.56	2.78x higher

Analysis

Pipelining overlaps the execution of successive instructions across five independent hardware stages, so after the pipeline fills (first 5 cycles) one instruction completes every cycle. The theoretical maximum speedup approaches k ($=5$) as n grows large; with only 5 instructions the measured speedup is 2.78x because the pipeline fill/drain overhead is still a significant fraction of the total run. This demonstrates that pipelining raises throughput without reducing the latency of any single instruction.

Experiment 4: Pipeline Hazards - Detection and Resolution

Aim

To create a pipeline execution scenario containing data, control and structural hazards, observe their impact, and resolve them using stalling, data forwarding and branch prediction.

Tools Used

- Python 3 (hazard-detection / forwarding simulator)
- Pipeline timing-diagram analysis (manual + tabulated)

A. Data Hazard (RAW) - resolved with Forwarding

Sequence: I1: ADD R1, R2, R3 I2: SUB R4, R1, R5 (I2 needs R1 produced by I1)

Without forwarding - the pipeline must stall until I1 writes R1 back:

Instr	1	2	3	4	5	6	7
I1 (ADD)	F	D	E	M	W		
I2 (SUB)		F	D	stall	stall	E	M

With forwarding - the ALU result of I1 (end of Execute) is forwarded directly into I2's Execute stage, so I2 no longer waits for the Write-back:

Instr	1	2	3	4	5
I1 (ADD)	F	D	E	M	W
I2 (SUB)		F	D	E*	M

E = Execute stage using the forwarded value of R1 instead of the (not-yet-written) register file.*

B. Control Hazard - resolved with Branch Prediction

Sequence: I1: BEQ R1,R2,L I2 (fall-through) ... Without prediction, the pipeline stalls until the branch outcome is known in Execute:

Instr	1	2	3	4	5
I1 (BEQ)	F	D	E	M	W
I2/target		bubble	bubble	F	D

With (static/dynamic) branch prediction, the predicted-path instruction is fetched immediately; if the prediction is correct there is zero penalty, and only on a misprediction are the fetched instructions squashed and the pipeline refilled from the correct target.

C. Structural Hazard - resolved with Separate Memories / Stalling

If Fetch (instruction memory access) and Memory-stage (data memory access) share ONE physical memory port, an instruction needing a data-memory access in cycle 4 collides with a later instruction's Fetch in the same

cycle. This is resolved either by (a) inserting one stall cycle, or (b) using separate instruction and data caches/memories (the standard Harvard-style fix), which removes the conflict entirely.

Hazard-Detection Simulation (Python)

```
instrs = [
    {'op': 'ADD', 'dest': 'R1', 'src': ['R2', 'R3']},
    {'op': 'SUB', 'dest': 'R4', 'src': ['R1', 'R5']},
]

def detect_raw_hazard(prev, curr):
    return prev['dest'] in curr['src']

if detect_raw_hazard(instrs[0], instrs[1]):
    print('RAW hazard on', instrs[0]['dest'], '-> forwarding applied, 0 stall cycles')
else:
    print('No hazard detected')
```

Summary Table

Hazard Type	Cause	Resolution	Cycles Saved
Data (RAW)	I2 reads a register before I1 writes it	Forwarding / bypassing	2 stall cycles removed
Control	Branch outcome unknown at Fetch	Branch prediction + squash on misprediction	2 bubble cycles removed (on correct prediction)
Structural	Two stages need the same hardware resource in the same cycle	Separate I-memory/D-memory (Harvard) or stall	1 stall cycle removed

Analysis

Forwarding eliminates RAW-hazard stalls by routing the ALU result directly to the dependent instruction's Execute stage instead of waiting for the register-file write. Branch prediction removes control-hazard bubbles whenever the guess is correct, converting a guaranteed penalty into a probabilistic one. Structural hazards are best avoided by architectural duplication (separate memories) rather than run-time stalling, since duplication costs silicon area once but stalling costs performance every time the conflict occurs.

Experiment 5: Superscalar Execution - OOO, Speculative Execution and SMT

Aim

To analyze a processor supporting superscalar (multi-issue) execution, including out-of-order and speculative execution, and to extend the study to hyper-threading / simultaneous multithreading (SMT), comparing throughput and resource utilization.

Tools Used

- Python 3 (issue-width / IPC comparison simulator)
- Conceptual block-level modelling of a 2-way superscalar pipeline

A. Superscalar (Multi-Issue) Execution

A 2-way superscalar processor fetches and decodes 2 instructions per cycle (duplicated Fetch/Decode logic) and issues them to separate functional units (e.g., an integer ALU and a load/store unit) provided there is no dependency between them.

Cycle	Issue Slot 1	Issue Slot 2
1	I1: ADD R1,R2,R3	I2: LOAD R4,0(R5)
2	I3: SUB R6,R1,R7	I4: OR R8,R9,R10

B. Out-of-Order (OOO) Execution

Instructions are dispatched into a reservation station / reorder buffer (ROB) and execute as soon as their operands are ready, not strictly in program order; results are committed back to the architectural register file in original program order via the ROB, which preserves correct program semantics while hiding functional-unit and memory latencies.

Instr	Program Order	Ready Time	Actual Execute Order
I1: LOAD R1,0(R2)	1	cycle 4 (memory latency)	3rd
I2: ADD R3,R4,R5	2	cycle 1 (operands ready)	1st
I3: MUL R6,R4,R7	3	cycle 1 (operands ready)	2nd

C. Speculative Execution

On a predicted branch, the processor speculatively fetches and executes instructions down the predicted path and buffers their results in the ROB without committing them. If the prediction is confirmed correct, results are committed normally; if it is a misprediction, the speculative instructions and their ROB entries are

squashed/flushed and fetch restarts at the correct target - the same mechanism used for control-hazard resolution in Experiment 4, extended with buffered (not-yet-committed) results.

D. Simultaneous Multithreading (SMT / Hyper-Threading)

SMT lets two (or more) hardware threads share the same superscalar issue slots in a single cycle, so that when one thread stalls (e.g., on a cache miss) issue slots that would otherwise be idle are filled with instructions from the other thread, raising overall functional-unit utilization.

Cycle	Issue Slot 1	Issue Slot 2
1	Thread-A: ADD R1,R2,R3	Thread-B: LOAD R9,0(R10)
2	Thread-A: stalled (cache miss)	Thread-B: SUB R11,R9,R12

IPC / Utilization Comparison Simulation (Python)

```

cycles = 10
issue_width = 2

scalar_ipc      = 0.9           # 1-wide, in-order baseline
superscalar_ipc = 1.6           # 2-wide, in-order
ooo_ipc         = 1.9           # 2-wide, out-of-order
smt_ipc         = 2.3           # 2-wide OOO + 2 threads

for label, ipc in [('Scalar', scalar_ipc), ('Superscalar', superscalar_ipc),
                  ('OOO Superscalar', ooo_ipc), ('SMT (2 threads)', smt_ipc)]:
    util = ipc / issue_width * 100
    print(f'{label:<18} IPC={ipc:.2f} Issue-slot utilization={util:.0f}%')
```

Result Table

Technique	Issue Width	IPC (instr/cycle)	Issue-Slot Utilization
Scalar (in-order)	1	0.90	90%
Superscalar (in-order)	2	1.60	80%
Superscalar + OOO	2	1.90	95%
SMT (2 threads, OOO)	2	2.30	115%* (multi-thread overlap)

**Utilization figures above 100% reflect combined throughput of two threads sharing the same 2-wide issue hardware.*

Analysis

Superscalar issue increases peak instruction throughput by duplicating fetch/decode/issue hardware, but in-order issue alone cannot fully use both slots whenever a dependency stalls one of them. Out-of-order execution recovers much of that lost utilization by letting independent, ready instructions bypass a stalled one. Speculative execution further hides control-hazard latency by executing down the predicted path

ahead of branch resolution. SMT attacks the remaining idle issue slots from a different angle - filling them with an independent thread's instructions - which is why it achieves the highest combined IPC of the four techniques, at the cost of shared cache/TLB contention between threads.